

Django Vue Lab

Занятие 3. router, view, form, CBW

Черепанов И.Н. Филиппьев В.А.

Менеджер URL-ов

```
from django.urls import path
from . import views

urlpatterns = [
    path('articles/2003/', views.special_case_2003),
    path('articles/<int:year>', views.year_archive),
    path('articles/<int:year>/<int:month>',
views.month_archive),
    path('articles/<int:year>/<int:month>/<int:pk>',
views.article_detail),
]
```

re_path

re_path нужен, когда стандартных converters в path() недостаточно и маршрут удобнее описать регулярным выражением.

```
from django.urls import re_path
from . import views

urlpatterns = [
    re_path(r'^articles/(?P<year>[0-9]{4})/$',
views.year_archive),
    re_path(r'^articles/(?P<year>[0-9]{4})/(?P<slug>[-\w]+)/$',
views.article_detail),
]
```

re_path: ограничения формата

```
from django.urls import re_path
from . import views

urlpatterns = [
    re_path(
        r'^reports/(?P<year>20[0-9]{2})/(?P<month>0[1-9]|'
        1[0-2])/$',
        views.month_report,
        name='month-report',
    ),
]
```

```
<a href="{% url 'month-report' year=2026 month='05' %}">Отчёт  
за май</a>
```

re_path: перехват всех URL

Catch-all маршрут должен идти последним: иначе он перехватит URL, которые должны обрабатываться более конкретными маршрутами.

```
from django.urls import path, re_path
from . import views

urlpatterns = [
    path('', views.index, name='index'),
    path('articles/<int:year>/', views.year_archive,
name='year-archive'),

    re_path(r'^(?P<path>.*)$', views.fallback,
name='fallback'),
    path( "<path:route>", views.fallback, name="index"),
]
```

re_path: перехват всех URL (ii)

```
from django.http import HttpResponseRedirect

def fallback(request, path):
    return HttpResponseRedirect(f'Страница не найдена: {path}')
```

Именованные группы

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path('articles/<int:year>/<int:month>/<int:day>/',
        views.article_detail),
]
```

```
views.article_detail(request, year=2003, month=3, day=3)
```

Необязательные параметры

```
from django.urls import path

from . import views

urlpatterns = [
    path('blog/', views.page),
    path('blog/page/<int:num>/', views.page),
]

# View (in blog/views.py)
def page(request, num="1"):
    # Output the appropriate page of blog entries, according to
    # num.
    ...
```


include

```
from django.urls import include, path

urlpatterns = [
    # ... snip ...
    path('community/',
include('django_website.aggregator.urls')),
    path('contact/', include('django_website.contact.urls')),
    # ... snip ...
]
```

Дополнительные аргументы

```
from django.urls import path
from . import views

urlpatterns = [
    path('blog/<int:year>/', views.year_archive, {'foo':
'bar'}),
]
```

Именованные URL

```
from django.urls import path
from . import views

urlpatterns = [
    path('articles/<int:year>', views.year_archive,
name='news-year-archive'),
]
```

```
<a href="{% url 'news-year-archive' 2012 %}">2012 Archive</a>
```

```
from django.urls import reverse
from django.http import HttpResponseRedirect

def redirect_to_year(request):
    year = 2006
```

Именованные URL (ii)

```
return HttpResponseRedirect(reverse('news-year-archive',  
args=(year,)))
```

Формы

Django-форма описывает HTML-поля, преобразует строки из запроса в Python-значения и проверяет данные.

- GET – показать пустую форму
- POST – принять данные от пользователя
- `is_valid()` – запустить валидацию
- `cleaned_data` – безопасные нормализованные значения

Form: описание полей

```
from django import forms

class FeedbackForm(forms.Form):
    name = forms.CharField(label='Имя', max_length=100)
    email = forms.EmailField(label='Email')
    message = forms.CharField(
        label='Сообщение',
        widget=forms.Textarea,
        min_length=10,
    )
```

- Поле формы отвечает за валидацию и преобразование типа
- Виджет отвечает за HTML-представление поля

Form: обработка во view

```
from django.shortcuts import redirect, render
from .forms import FeedbackForm

def feedback(request):
    if request.method == 'POST':
        form = FeedbackForm(request.POST)
        if form.is_valid():
            send_feedback(form.cleaned_data)
            return redirect('feedback-ok')
    else:
        form = FeedbackForm()

    return render(request, 'feedback.html', {'form': form})
```

Form: шаблон

```
<form method="post">
  {% csrf_token %}

  {{ form.non_field_errors }}

  {% for field in form %}
    <div>
      {{ field.label_tag }}
      {{ field }}
      {{ field.errors }}
    </div>
  {% endfor %}

  <button type="submit">Отправить</button>
</form>
```


Form: кастомная валидация

```
from django import forms

class RegisterForm(forms.Form):
    username = forms.CharField(max_length=30)
    password = forms.CharField(widget=forms.PasswordInput)
    password_repeat =
forms.CharField(widget=forms.PasswordInput)

    def clean_username(self):
        username = self.cleaned_data['username']
        if username.lower() == 'admin':
            raise forms.ValidationError('Это имя занято')
        return username

    def clean(self):
        data = super().clean()
        if data.get('password') != data.get('password_repeat'):
```

Form: кастомная валидация (ii)

```
raise forms.ValidationError('Пароли не совпадают')  
return data
```

ModelForm

ModelForm строит форму по модели и умеет сохранять объект.

```
from django import forms
from .models import Article

class ArticleForm(forms.ModelForm):
    class Meta:
        model = Article
        fields = ['title', 'text', 'published_at']
```

```
def article_create(request):
    form = ArticleForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        article = form.save()
        return redirect('article-detail', pk=article.pk)
    return render(request, 'article_form.html', {'form': form})
```

Общие классы представлений

Меньше кода – меньше проблем

```
# some_app/views.py
from django.views.generic import ListView, TemplateView
from .models import Publisher

class AboutView(TemplateView):
    template_name = "about.html"

class PublisherList(ListView):
    model = Publisher
```

```
# urls.py
from django.urls import path
from some_app.views import AboutView
```

Общие классы представлений (ii)

```
urlpatterns = [  
    path('about/', AboutView.as_view(), name='about'),  
]
```

Шаблон ListView

```
{% extends "base.html" %}

{% block content %}
    <h2>Publishers</h2>
    <ul>
        {% for publisher in object_list %}
            <li>{{ publisher.name }}</li>
        {% endfor %}
    </ul>
{% endblock %}
```

DetailView

```
from django.views.generic import DetailView
from books.models import Publisher, Book

class PublisherDetail(DetailView):
    model = Publisher

    def get_context_data(self, **kwargs):
        # Call the base implementation first to get a context
        context = super().get_context_data(**kwargs)
        # Add in a QuerySet of all the books
        context['book_list'] = Book.objects.all()
        return context
```

ListView

```
from django.views.generic import ListView
from books.models import Book

class AcmeBookList(ListView):
    context_object_name = 'book_list'
    queryset = Book.objects.filter(publisher__name='Acme
Publishing')
    template_name = 'books/acme_list.html'
```


ListView: get_queryset

```
# urls.py
from django.urls import path
from books.views import PublisherBookList

urlpatterns = [
    path('books/<slug:publisher_slug>/',
    PublisherBookList.as_view(), name='publisher-books'),
]
```

```
# views.py
from django.shortcuts import get_object_or_404
from django.views.generic import ListView
from books.models import Book, Publisher

class PublisherBookList(ListView):
    template_name = 'books/books_by_publisher.html'
```

ListView: get_queryset (ii)

```
def get_queryset(self):
    self.publisher = get_object_or_404(Publisher,
slug=self.kwargs['publisher_slug'])
    return Book.objects.filter(publisher=self.publisher)

def get_context_data(self, **kwargs):
    # Call the base implementation first to get a context
    context = super().get_context_data(**kwargs)
    # Add in the publisher
    context['publisher'] = self.publisher
    return context
```

CreateView, UpdateView, DeleteView

```
from django.views.generic.edit import CreateView, UpdateView, DeleteView
from django.urls import reverse_lazy
from myapp.models import Author

class AuthorCreate(CreateView):
    model = Author
    fields = ['name']

class AuthorUpdate(UpdateView):
    model = Author
    fields = ['name']

class AuthorDelete>DeleteView):
    model = Author
    success_url = reverse_lazy('author-list')
```

form_valid

```
from django.views.generic.edit import CreateView
from myapp.models import Author

class AuthorCreate(CreateView):
    model = Author
    fields = ['name']

    def form_valid(self, form):
        form.instance.created_by = self.request.user
        return super().form_valid(form)
```

Примеси

```
from django.http import JsonResponse

class AjaxableResponseMixin:
    def form_invalid(self, form):
        response = super().form_invalid(form)
        if self.request.headers.get('x-requested-with') ==
'XMLHttpRequest':
            return JsonResponse(form.errors, status=400)
        return response

    def form_valid(self, form):
        response = super().form_valid(form)
        if self.request.headers.get('x-requested-with') ==
'XMLHttpRequest':
            data = {
                'pk': self.object.pk,
            }
```

Примеси (ii)

```
        return JsonResponse(data)  
    return response
```

Примеси: использование

```
class AuthorCreate(AjaxableResponseMixin, CreateView):  
    model = Author  
    fields = ['name']
```

Основные классы

- TemplateView
- RedirectView
- DetailView
- FormView
- CreateView
- UpdateView
- DeleteView
- и прочие

TemplateView

Атрибуты:

- `content_type`
- `http_method_names`
- `response_class` [`render_to_response()`]
- `template_engine`
- `template_name` [`get_template_names()`]

Методы:

- `as_view()`
- `dispatch()`
- `get()`
- `get_context_data()`
- `head()`
- `http_method_not_allowed()`
- `render_to_response()`

DetailView

Атрибуты:

- `context_object_name` [`get_context_object_name()`]
- `model`
- `pk_url_kwarg`
- `queryset` [`get_queryset()`]
- `response_class` [`render_to_response()`]
- `slug_field` [`get_slug_field()`]
- `slug_url_kwarg`
- `template_engine`
- `template_name` [`get_template_names()`]
- `template_name_field`
- `template_name_suffix`

Методы:

- `get_object()`

ListView

Атрибуты:

- ordering [get_ordering()]
- paginate_by [get_paginate_by()]
- paginator_class
- queryset [get_queryset()]
- response_class [render_to_response()]

Методы:

- paginate_queryset()
- get_paginator()

FormView

Атрибуты:

- `form_class` [`get_form_class()`]
- `http_method_names`
- `initial` [`get_initial()`]
- `success_url` [`get_success_url()`]
- `template_name` [`get_template_names()`]

Методы:

- `as_view()`
- `dispatch()`
- `form_invalid()`
- `form_valid()`
- `get()`
- `get_context_data()`
- `get_form()`

FormView (ii)

- `get_form_kwargs()`
- `http_method_not_allowed()`
- `post()`
- `put()`

Правила обработки форм

Не доверяйте пользователю. Любые данные из браузера считаются недостоверными, даже если форма была сгенерирована сервером.

- Все проверки должны выполняться на сервере
- Клиентская валидация удобна, но не является защитой
- Используйте `form.is_valid()`, `cleaned_data` и методы `clean()`
- Не используйте `request.POST` напрямую для бизнес-логики

Формы: CSRF

CSRF защищает от отправки формы от имени пользователя с чужого сайта.

```
<form method="post">
  {% csrf_token %}
  {{ form.as_p }}
  <button type="submit">Сохранить</button>
</form>
```

- В HTML-формах с POST всегда добавляйте {% csrf_token %}
- Не отключайте CSRF без необходимости
- Для API используйте явную схему аутентификации и защиты

Формы: ошибки

Ошибки валидации нужно показывать пользователю рядом с формой, а не терять или заменять общей ошибкой.

```
def feedback(request):  
    if request.method == 'POST':  
        form = FeedbackForm(request.POST)  
        if form.is_valid():  
            send_feedback(form.cleaned_data)  
            return redirect('feedback-ok')  
    else:  
        form = FeedbackForm()  
  
    return render(request, 'feedback.html', {'form': form})
```

При невалидной форме тот же объект `form` возвращается в шаблон: в нём уже есть введённые значения и ошибки.

Формы: инъекции

Инъекция возникает, когда пользовательский ввод попадает в команду, SQL, HTML или путь к файлу как код, а не как данные.

- Не собирайте SQL строками: используйте ORM или параметры запроса
- Не вставляйте пользовательский HTML без очистки
- Не передавайте ввод пользователя в shell-команды
- Проверяйте типы, диапазоны, длину и допустимые значения
- Экранирование в шаблонах Django включено по умолчанию, не отключайте его без причины